Abstract Data Types — Comprehensive Reference

Every ADT below follows the same structure: Definition → Operations & Complexity → Pros/Cons → Use Cases → Real-World Applications → Hybrid Structures.

1. ARRAY

Definition

A fixed-size, contiguous block of memory storing elements of the same type, accessed via index in O(1) using base_address + (index * element_size).

Operations & Complexity

Operation	Best	Average	Worst
Access by index	O(1)	O(1)	O(1)
Search (unsorted)	O(1)	O(n)	O(n)
Search (sorted, binary)	O(1)	O(log n)	O(log n)
Insert at end (with space)	O(1)	O(1)	O(1)
Insert at beginning/middle	O(n)	O(n)	O(n)
Delete at end	O(1)	O(1)	O(1)
Delete at beginning/middle	O(n)	O(n)	O(n)

Pros & Cons

Pros: O(1) random access; cache-friendly (contiguous memory, great spatial locality); simple, low overhead.

Cons: Fixed size (static arrays); expensive insert/delete in middle (shifting required); resizing a dynamic array costs O(n) amortized.

Use Cases

Lookup tables, matrices, image pixel buffers, implementing other structures (heaps, hash tables) internally.

Real-World Applications

Image processing (pixel grids), spreadsheets, database record storage, game boards, audio sample buffers.

Hybrid Structures

Dynamic Array (amortized O(1) append via doubling — e.g. C++ vector, Python list, Java ArrayList); Array-backed Heap; Array-backed Hash Table (open addressing).

2. LINKED LIST (Singly, Doubly, Circular)

Definition

A sequence of nodes where each node holds data and a pointer/reference to the next (and previous, for doubly) node. No contiguous memory requirement.

Operations & Complexity

Operation	Best	Average	Worst
Access by index	O(1)	O(n)	O(n)

Search	O(1)	O(n)	O(n)
Insert at head	O(1)	O(1)	O(1)
Insert at tail (no tail ptr)	O(n)	O(n)	O(n)
Insert at tail (with tail ptr)	O(1)	O(1)	O(1)
Delete at head	O(1)	O(1)	O(1)
Delete (given node ref, doubly)	O(1)	O(1)	O(1)

Pros & Cons

Pros: O(1) insert/delete at known position (no shifting); dynamic size, no resizing cost; efficient memory use when size is unpredictable.

Cons: No random access (must traverse); extra memory per node (pointer overhead); poor cache locality (nodes scattered in memory).

Use Cases

Implementing stacks/queues/deques; undo-redo history; music/playlist navigation (circular); managing free memory blocks in allocators.

Real-World Applications

Browser back/forward history, music player playlists (circular), OS process scheduling (circular round-robin), LRU cache (doubly linked list + hashmap).

Hybrid Structures

LRU Cache (HashMap + Doubly Linked List); Skip List (linked list + multiple express lanes, used in Redis); Unrolled Linked List (linked list of small arrays for cache efficiency).

3. STACK (LIFO)

Definition

Last-In-First-Out structure supporting push (add to top) and pop (remove from top). Can be implemented via array or linked list.

Operations & Complexity

Operation	Best	Average	Worst
Push	O(1)	O(1)	O(1)*
Pop	O(1)	O(1)	O(1)
Peek/Top	O(1)	O(1)	O(1)
Search	O(1)	O(n)	O(n)

*Array-based push is O(n) worst-case only during resize (amortized O(1)).

Pros & Cons

Pros: Simple, fast O(1) core operations; natural fit for nested/recursive problems; minimal memory overhead.

Cons: No random access; fixed capacity if array-backed without resizing; only top element accessible.

Use Cases

Expression evaluation/parsing (balanced parentheses, postfix conversion), function call management (call stack), undo mechanisms, DFS traversal.

Real-World Applications

Compiler syntax checking, browser back button, undo/redo in text editors, recursive algorithm execution (system call stack).

Hybrid Structures

Min/Max Stack (stack + auxiliary stack tracking min/max in O(1)); Two Stacks in One Array; Stack-based Queue (two stacks simulating FIFO).

4. QUEUE (FIFO)

Definition

First-In-First-Out structure supporting enqueue (add to rear) and dequeue (remove from front). Variants: circular queue, deque, priority queue.

Operations & Complexity

Operation	Best	Average	Worst
Enqueue	O(1)	O(1)	O(1)*
Dequeue	O(1)	O(1)	O(1)
Peek/Front	O(1)	O(1)	O(1)
Search	O(1)	O(n)	O(n)

*Naive array-based queue has O(n) dequeue due to shifting; circular buffer or linked-list implementation fixes this to true O(1).

Pros & Cons

Pros: Fair, ordered processing; O(1) core ops with proper implementation (circular buffer/linked list); natural fit for sequential/level-based processing.

Cons: No random access; naive array implementation wastes space or requires shifting; basic queue can't prioritize (need priority queue for that).

Use Cases

BFS traversal (shortest path, level-order tree traversal), task scheduling, request buffering, print job management.

Real-World Applications

CPU/OS task scheduling, printer job queues, customer service call centers, message brokers (Kafka, RabbitMQ), web server request handling.

Hybrid Structures

Circular Buffer (array-based queue reusing freed space, used in streaming/audio buffers); Deque (double-ended queue, array or doubly-linked-list backed); Priority Queue (queue + heap ordering).

5. PRIORITY QUEUE / HEAP

Definition

An ADT where each element has a priority; highest (or lowest) priority is served first. Typically implemented as a Binary Heap (complete binary tree stored in an array).

Operations & Complexity

Operation	Best	Average	Worst

Insert	O(1)	O(log n)	O(log n)
Extract Min/Max	O(log n)	O(log n)	O(log n)
Peek Min/Max	O(1)	O(1)	O(1)
Build Heap (from array)	O(n)	O(n)	O(n)
Decrease/Increase Key	O(1)	O(log n)	O(log n)
Search arbitrary element	O(1)	O(n)	O(n)

Pros & Cons

Pros: O(log n) insert/extract is excellent for repeated min/max queries; O(1) peek; array-backed = cache-friendly, no pointer overhead.

Cons: No O(1) search for arbitrary elements; not fully sorted (only root guaranteed); not stable for equal priorities by default.

Use Cases

Dijkstra's/Prim's algorithms, task scheduling by priority, top-K elements problems, median maintenance (two-heap technique), event simulation.

Real-World Applications

OS process scheduling by priority, Dijkstra's shortest path (Google Maps routing), Huffman coding, hospital emergency triage systems, A* pathfinding in games.

Hybrid Structures

Two-Heap Median Finder (max-heap + min-heap for running median); Fibonacci Heap (used in advanced Dijkstra/Prim implementations, faster decrease-key); Indexed Priority Queue (heap + hashmap for O(log n) arbitrary updates).

6. HASH TABLE / HASH MAP

Definition

Maps keys to values using a hash function to compute an index into an underlying array (bucket array). Collisions handled via chaining (linked lists/trees per bucket) or open addressing (probing).

Operations & Complexity

Operation	Best	Average	Worst
Insert	O(1)	O(1)	O(n)*
Search/Get	O(1)	O(1)	O(n)*
Delete	O(1)	O(1)	O(n)*

*Worst case occurs with poor hash function causing all keys to collide into one bucket.

Pros & Cons

Pros: O(1) average for all core operations; flexible key types; widely supported and optimized in most languages.

Cons: No ordering of keys; worst-case O(n) with bad hashing/collisions; resizing causes O(n) rehash cost; memory overhead from load-factor management.

Use Cases

Fast lookups (caching, memoization), counting frequencies, deduplication, implementing sets, database indexing, symbol tables in compilers.

Real-World Applications

Database indexing, DNS lookup, password storage (hashed), spell checkers, caching layers (Redis is essentially a giant hash map), compiler symbol tables.

Hybrid Structures

LRU Cache (HashMap + Doubly Linked List); Hash Map with Tree Buckets (Java 8+ HashMap converts long chains to Red-Black Trees for O(log n) worst case); Bloom Filter (probabilistic hash-based set, used for quick existence checks at scale).

7. BINARY SEARCH TREE (BST)

Definition

A binary tree where left subtree < node < right subtree for every node, enabling O(log n) search in a balanced tree (degrades to O(n) if unbalanced/skewed).

Operations & Complexity (Balanced / Unbalanced-Worst)

Operation	Best	Average	Worst
Search	O(1)	O(log n)	O(n)
Insert	O(1)	O(log n)	O(n)
Delete	O(1)	O(log n)	O(n)
Find Min/Max	O(log n)	O(log n)	O(n)
In-order Traversal	O(n)	O(n)	O(n)

Pros & Cons

Pros: O(log n) ordered operations when balanced; in-order traversal gives sorted output; supports range queries and predecessor/successor naturally.

Cons: Degrades to O(n) (linked-list-like) if inserted in sorted order without rebalancing; pointer overhead; more complex than array/hash for simple lookups.

Use Cases

Maintaining sorted, dynamic data with frequent insert/delete; range queries; implementing ordered maps/sets; expression trees.

Real-World Applications

File system directory structures, database indexing (B-Trees are a generalization), auto-complete/dictionary lookups, syntax trees in compilers.

Hybrid Structures

AVL Tree (BST + self-balancing via rotations, guarantees O(log n)); Red-Black Tree (BST + color-based balancing, used in C++ map/set, Java TreeMap); B-Tree/B+ Tree (multi-way balanced BST, used in databases/filesystems).

8. TRIE (Prefix Tree)

Definition

A tree where each path from root to a node represents a prefix of stored strings; each node typically has up to 26 (alphabet size) children pointers.

Operations & Complexity (m = key length)

Operation	Best	Average	Worst

Insert	O(m)	O(m)	O(m)
Search	O(m)	O(m)	O(m)
Prefix Search	O(m)	O(m)	O(m)
Delete	O(m)	O(m)	O(m)

Pros & Cons

Pros: Extremely fast prefix-based search, independent of number of stored words; no hash collisions; supports autocomplete naturally.

Cons: High memory usage (each node can have many child pointers, mostly empty); not cache-friendly (scattered pointers); overkill for simple exact-match-only lookups.

Use Cases

Autocomplete/typeahead search, spell checking, IP routing (longest prefix match), word games (Boggle, Scrabble validation).

Real-World Applications

Google search autocomplete, mobile keyboard predictive text, router IP address lookup tables, dictionary/spell-check applications.

Hybrid Structures

Compressed Trie / Radix Tree (merges single-child chains to save memory, used in IP routing tables); Ternary Search Tree (BST + Trie hybrid, lower memory than standard trie); Suffix Tree (trie of all suffixes, used in string matching/bioinformatics).

9. GRAPH

Definition

A set of vertices (nodes) connected by edges, representing relationships. Can be directed/undirected, weighted/unweighted. Stored via Adjacency Matrix (V x V array) or Adjacency List (array/map of lists).

Operations & Complexity (V = vertices, E = edges)

Operation	Best	Average	Worst
Add Vertex (adj. list)	O(1)	O(1)	O(1)
Add Edge (adj. list)	O(1)	O(1)	O(1)
Check edge exists (matrix)	O(1)	O(1)	O(1)
Check edge exists (list)	O(1)	O(degree)	O(V)
BFS / DFS traversal	O(V+E)	O(V+E)	O(V+E)
Dijkstra (with min-heap)	-	O((V+E) log V)	O((V+E) log V)

Pros & Cons

Pros: Models complex many-to-many relationships naturally; adjacency list is memory-efficient for sparse graphs; supports rich algorithms (shortest path, MST, flow).

Cons: Adjacency matrix wastes O(V^2) memory for sparse graphs; many graph algorithms are computationally expensive (NP-hard problems like TSP); cycle detection/traversal complexity grows with density.

Use Cases

Social networks (friend connections), road/transportation networks, recommendation systems, dependency resolution (build systems, package managers), network topology.

Real-World Applications

Google Maps/GPS routing (Dijkstra/A*), social media friend suggestions, Facebook/LinkedIn connection graphs, package dependency managers (npm, pip), airline route planning.

Hybrid Structures

Weighted Graph + Priority Queue (Dijkstra's algorithm); Graph + Union-Find/Disjoint Set (Kruskal's MST, cycle detection); Graph + HashMap (adjacency list using hashmap for O(1) average edge lookup).

10. DISJOINT SET / UNION-FIND

Definition

A structure tracking a partition of elements into disjoint (non-overlapping) sets, supporting union (merge two sets) and find (identify which set an element belongs to).

Operations & Complexity (with Path Compression + Union by Rank)

Operation	Best	Average	Worst
Find	O(1)	$O(\alpha(n)) \approx O(1)$	$O(\alpha(n)) \approx O(1)$
Union	O(1)	$O(\alpha(n)) \approx O(1)$	$O(\alpha(n)) \approx O(1)$

$\alpha(n)$ = inverse Ackermann function, grows so slowly it's effectively constant for all practical input sizes.

Pros & Cons

Pros: Near-constant time operations with optimizations; extremely simple and memory-efficient (just a parent array); ideal for connectivity problems.

Cons: Only answers 'same set?' not full relationship structure; no way to split sets back apart (union is one-directional); not useful for general graph traversal needs.

Use Cases

Cycle detection in graphs, network connectivity, Kruskal's Minimum Spanning Tree, image segmentation (connected components), social network 'friend circles'.

Real-World Applications

Network connectivity monitoring, image processing (connected component labeling), Kruskal's MST in network design (minimum cost wiring/cabling), percolation theory simulations.

Hybrid Structures

Union-Find + Kruskal's Algorithm (MST construction); Weighted Union-Find (union by size/rank for better balance); Union-Find with Rollback (used in offline dynamic connectivity problems).

Master Summary Table

ADT	Search	Insert	Delete	Key Strength
Array	O(n)/O(log n)	O(n)	O(n)	Random access O(1)
Linked List	O(n)	O(1)	O(1)	O(1) insert/delete at known pos
Stack	O(n)	O(1)	O(1)	LIFO order, recursion
Queue	O(n)	O(1)	O(1)	FIFO order, BFS
Heap	O(n)	O(log n)	O(log n)	Min/Max in O(1)

Hash Table	O(1) avg	O(1) avg	O(1) avg	Fastest average lookup
BST (balanced)	O(log n)	O(log n)	O(log n)	Sorted + dynamic
Trie	O(m)	O(m)	O(m)	Prefix search
Graph	O(V+E)	O(1)	O(V+E)	Relationships/networks
Union-Find	O(α(n))	O(α(n))	-	Connectivity queries